D0 High-Level Design: Smart Barbell Load Cuffs

Assignment 4, Part 2 · Design D0
Team: Tyler Beck, Colin Hill
Source files (this folder): `D0_Block_Diagram.drawio`, `D0_Data_Flow.drawio` (open in diagrams.net), SVG exports of both, and this document.

1. Title, goal statement, and conventions

Title: Smart Barbell Load Cuffs

Goal statement: Log every set's weight, reps, and bar speed to the right lifter's account automatically, starting from one phone tap on the bar.

Basic input: plates loaded on a standard Olympic barbell, the bar's motion during a set, and a lifter tapping their phone on the bar.
Basic output: a confirmed set record (total weight, rep count, speed of every rep) stored in that lifter's account, plus staff views of sets and bar status.

Conventions (the same legend is printed on each diagram)

Symbol	Meaning
White box, solid blue border, tag **C#**	A component our team builds
Grey box, dashed border, tag **X#**	An external system we depend on but do not build
Light dashed region	An external platform boundary; the components drawn inside it run on that platform
Yellow rounded box	A person (actor)
Brown box	A physical object (force, not data)
Solid arrow	Data moving one way; the head shows the direction
Double-headed arrow	Request and reply in both directions
Yellow double arrow	A person interacting with an app (screen, touch, vibration)
Brown dotted arrow	Physical force or strain
Grey dashed arrow	Planned for phase 2, not built this term
I# label on a line	Interface ID, defined in the interface specification table (part 4)
Data-flow diagram only	Rounded box **P#** = process (says which component runs it); open-ended box **D#** = data store; red box = timing budget set by a requirement

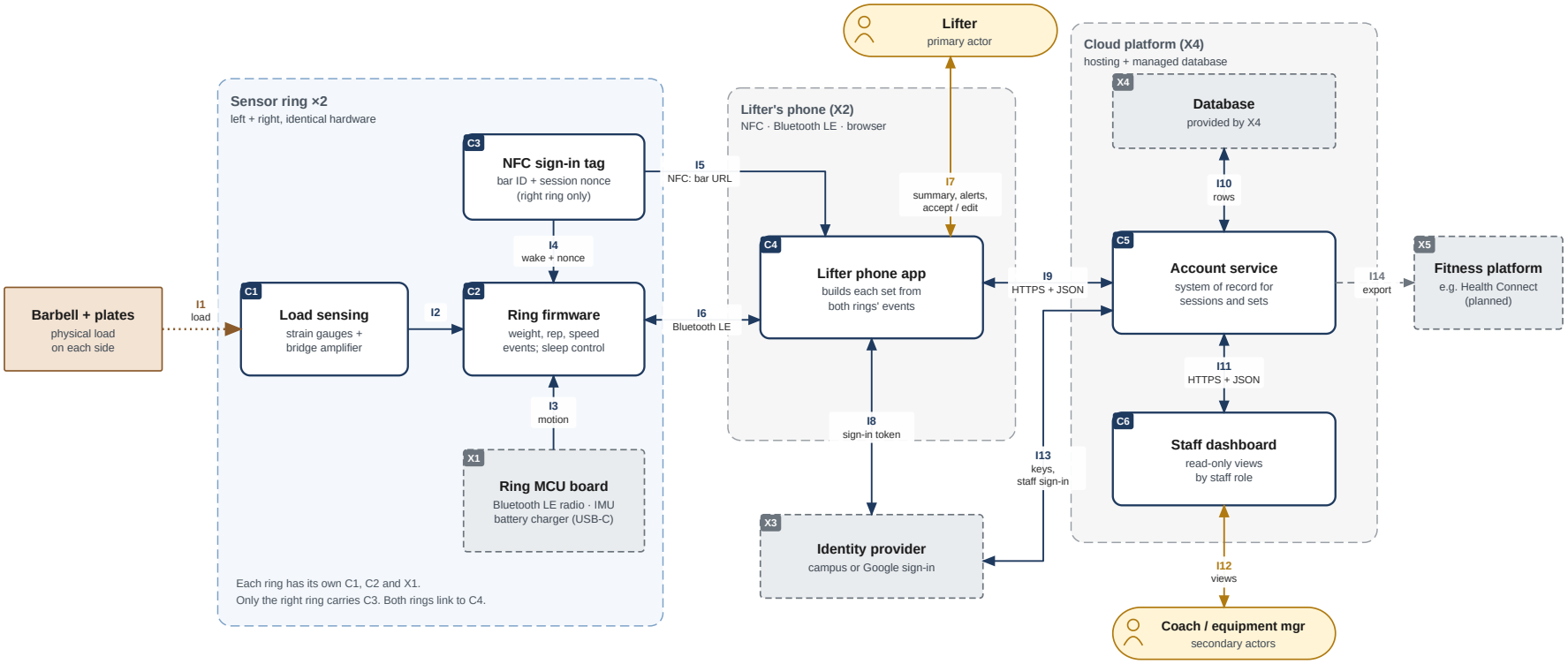
2. Block diagram (the D0 diagram)

Smart Barbell Load Cuffs · D0 Block Diagram

Team: Tyler Beck, Colin Hill · D0 rev 1

Goal: log every set's weight, reps, and bar speed to the right lifter's account automatically, starting from one phone tap on the bar.

Basic input: plates loaded on the bar, bar motion, and a phone tap. Basic output: a confirmed set record in the lifter's account, plus staff views of sets and bar status.



Notes:

- There are two sensor rings, one inside each collar, with identical hardware. Each ring has its own C1, C2 and X1. Only the right ring carries the NFC tag (C3). Each ring connects to the phone on its own (I6), and the phone app adds the left and right sides together.
- Six components are built by the team (C1–C6). Five external systems are drawn in grey (X1–X5). X5 is planned for phase 2.

3. Component responsibility table

ID	Component	Responsibility (one sentence)	Interfaces in	Interfaces out	Owner	Stories
C1	Load sensing (strain gauges + bridge amplifier), one per ring	Converts the shear strain in the bar on its side into a digital count.	I1	I2	Tyler Beck	US-01
C2	Ring firmware, one per ring	Turns its ring's raw readings into timestamped weight, rep and speed events.	I2, I3, I4, I6 (commands)	I6 (events)	Tyler Beck	US-01, US-02, US-03, US-04
C3	NFC sign-in tag, right ring only	Gives a tapping phone the bar's identity and current session nonce.	—	I4, I5	Tyler Beck	US-01, US-04
C4	Lifter phone app	Builds each confirmed set for the signed-in lifter from both rings' events.	I5, I6, I7, I8, I9 (replies)	I6 (commands), I7, I9	Colin Hill	US-01, US-02, US-04
C5	Account service	Keeps the single authoritative record of sessions, sets and bar status.	I9, I10, I11 (queries), I13	I9 (replies), I10, I11, I14	Colin Hill	US-01, US-03, US-04
C6	Staff dashboard	Shows stored sets and bar status to signed-in staff according to their role.	I11	I12	Colin Hill	US-02, US-03

External systems (not built by us)

ID	External system	What we depend on it for	Interfaces
X1	Ring MCU board (Bluetooth LE radio, motion sensor, battery charger with USB-C)	Runs C2, supplies motion data, charges the ring's battery	I3, carries I6
X2	Lifter's Android phone (NFC reader, Bluetooth LE, browser)	Runs C4, reads the tag, provides the radio link to the rings	carries I5, I6, I7
X3	Identity provider (campus or Google sign-in)	Proves who the lifter or staff member is; we store no passwords	I8, I13
X4	Cloud platform (hosting + managed database)	Runs C5 and C6, stores all rows	I10
X5	Fitness platform, e.g. Health Connect (phase 2)	Receives exported workouts	I14

Every story traces to at least one component: US-01 (lifter logging) → C1–C5; US-02 (coach bar speed) → C2, C4, C6; US-03 (equipment manager, battery and reporting status) → C2, C5, C6; US-04 (privacy officer, one account per set) → C3, C4, C5.

4. Interface specification table

ID	From → To	Inputs → outputs (name: type)	Data format	Protocol	On error: what happens (handled by)
I1	Barbell + plates → C1	plate load per side: physical kg → bridge voltage: analog, µV/V (≈5 µV/V per 20 kg)	Analog differential signal on 4 wires (E+, E−, S+, S−)	Two ±45° shear rosettes bonded to the bar, wired as a full Wheatstone bridge	Gauge debonds or a wire breaks: the count goes to full scale or stops changing. **C2** flags `SENSOR_FAULT` in its status and stops sending weight.
I2	C1 → C2	raw_count: int24 (signed), data_ready: bool	24-bit two's complement, MSB first	2-wire synchronous serial (clock + data), 10 samples/s; C2 powers C1 down by holding the clock high	No data_ready within 200 ms: **C2** power-cycles C1 up to 3 times, then sets `SENSOR_FAULT`.
I3	X1 → C2	accel_xyz: int16×3 (mg), gyro_xyz: int16×3 (mdps), wake_irq: bool	Little-endian register block	I2C at 400 kHz, 104 samples/s while awake; wake-on-motion interrupt line while asleep	Motion sensor does not answer: **C2** resets it once; if it still fails, C2 reports `REPS_UNAVAILABLE` and keeps sending weight.
I4	C3 → C2	field_detect: bool (interrupt), nonce: uint32	Interrupt pulse; 4-byte register	Interrupt line + I2C read	Nonce read fails: **C2** keeps the previous nonce, so the BIND check on I6 fails and **C4** asks the lifter to tap again.
I5	C3 → C4 (read by X2)	bar_url: string → barId: string (6 chars), nonce: uint32 (hex)	NDEF URI record	NFC (ISO 15693), phone held within 3 cm	Partial read: the phone does nothing and the lifter taps again (**X2**). Unknown barId: **C4** shows "bar not registered".
I6	C2 ↔ C4	C2 → C4 events, each ending in set_seq: uint16: weight {side: L/R, kg: float32, still: bool}; rep {rep_no: uint8, v_mean: float16 m/s, v_peak: float16 m/s}; status {battery_pct: uint8, faults: uint8, stored_sets: uint8}. C4 → C2 commands: ZERO; BIND {session_id: uint32, nonce: uint32}; ACK {set_seq: uint16}	Binary packets of at most 12 bytes, little-endian, type byte first	Bluetooth LE GATT: one service with a notify characteristic (events) and a write characteristic (commands)	Link drops: **C2** keeps up to 20 un-ACKed sets in flash and resends them on reconnect; **C4** drops duplicates by set_seq. Wrong nonce on BIND: **C2** rejects it and **C4** asks for a new tap.
I7	C4 ↔ Lifter	Out: set summary {kg, reps, m/s per rep}, imbalance alert {side, kg difference}, "bar in use" notice. In: accept, edited rep count, end session	On-screen text plus vibration pattern	Touch screen and phone vibration	No response from the lifter: **C4** confirms the set automatically after 12 s of bar idle.
I8	C4 ↔ X3	Out: sign-in request. In: id_token: JWT string, expires_at: timestamp	JWT (JSON)	OpenID Connect over HTTPS	Token expired or rejected: **C4** sends the lifter to sign in again and queues sets locally meanwhile.
I9	C4 ↔ C5	`POST /sessions` {barId, nonce} → {sessionId}; `POST /sets` {set record} → {setId}; `POST /sessions/{id}/end`; `POST /bars/{id}/status` {battery_pct, faults}	JSON, UTF-8	HTTPS REST with a bearer token from I8	Offline or 5xx: **C4** queues and retries with backoff for up to 24 h. 409 `BAR_IN_USE`: **C4** shows "bar in use" (UC-02 E1.4). 401: **C4** re-runs I8. Repeated setId: **C5** returns the stored record unchanged (idempotent).

ID	From → To	Inputs → outputs (name: type)	Data format	Protocol	On error: what happens (handled by)
I10	C5 ↔ X4 database	Rows: lifter, session, set, rep, bar status	Relational rows	Database connection over TLS	Write fails: **C5** answers 503 on I9 and **C4** retries from its queue. Two open sessions for one bar are refused by a unique constraint (**X4**), which **C5** reports as 409.
I11	C6 ↔ C5	Query {teamId or barId, date range} + staff token → set rows, bar-status rows	JSON	HTTPS REST (GET)	Role not allowed: **C5** returns 403 and **C6** shows "no access". 5xx: **C6** shows the last data with "updated at" time.
I12	C6 ↔ Coach / equipment manager	Out: per-rep speed table per athlete; bar list {battery %, last report time, faults}. In: filters	Web page	Browser	Bar has not reported for 7 days: **C6** marks it "check bar".
I13	C5 ↔ X3	Out: key request, staff sign-in redirect. In: signing keys (JWKS), staff id_token	JSON Web Key Set, JWT	HTTPS (OpenID Connect)	X3 unreachable: **C5** uses keys cached for up to 24 h; with no cached keys it answers 503.
I14	C5 → X5 (planned)	Workout session records	X5's workout schema (JSON)	X5's API over HTTPS	Record rejected: **C5** logs it and retries at the next sync; never blocks I9.

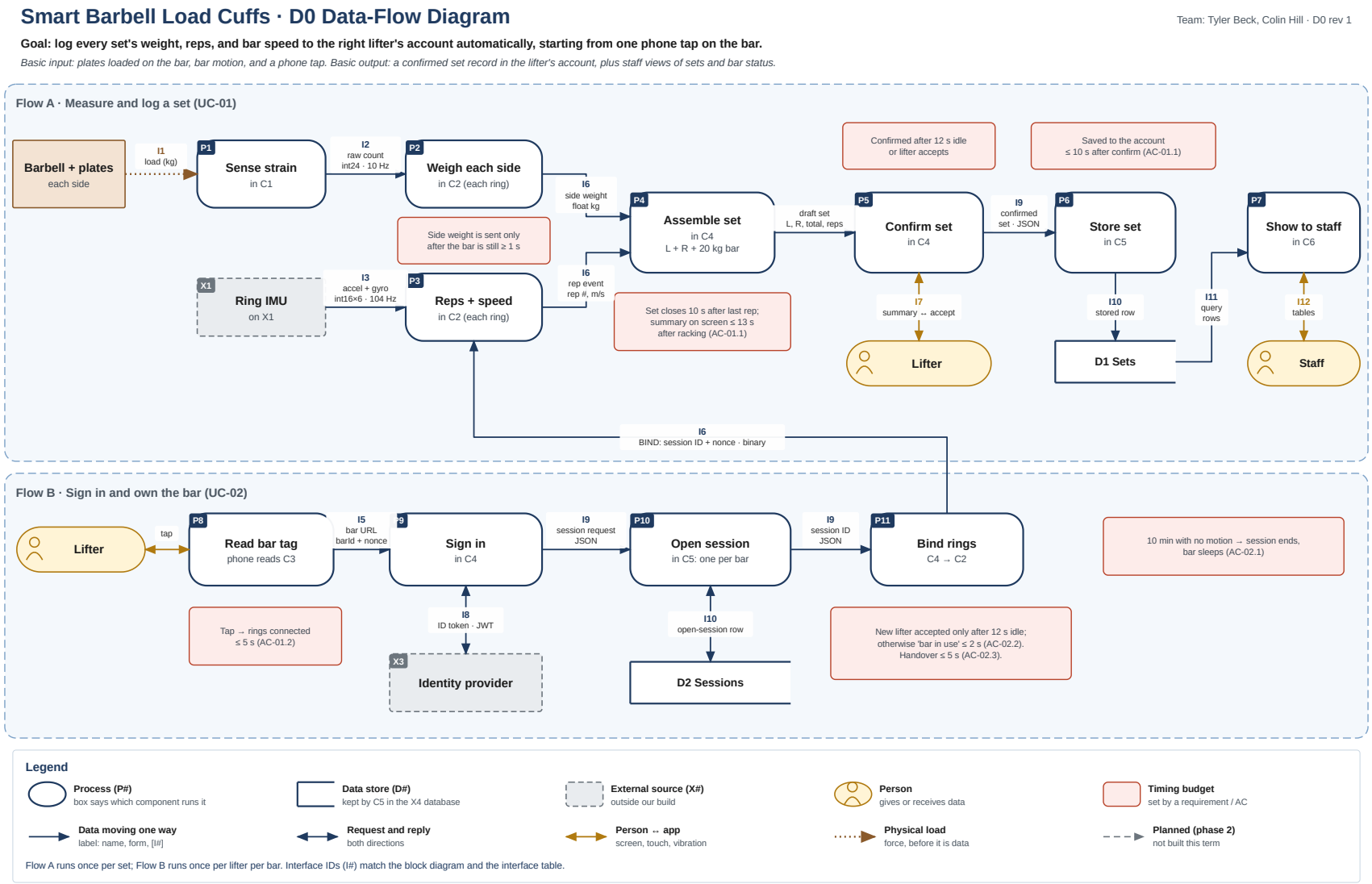
Example payload 1 — I6 weight event (C2 → C4), right side, 40.2 kg, bar still:

```
01 52 CD CC 20 42 01 07 00
│  │  └─ kg float32 = 40.2  │  └─ set_seq uint16 = 7
│  └─ side 'R'              └─ still = 1
└─ type 0x01 = weight
```

Example payload 2 — I9 `POST /sets` (C4 → C5):

```
{
  "setId": "R7Q2KX-0412-0007",
  "sessionId": 41207,
  "barId": "R7Q2KX",
  "leftKg": 40.1,
  "rightKg": 40.2,
  "barKg": 20.0,
  "totalKg": 100.3,
  "reps": [
    {"n": 1, "vMean": 0.62, "vPeak": 0.91},
    {"n": 2, "vMean": 0.58, "vPeak": 0.86}
  ],
  "edited": false,
  "endedAt": "2026-11-03T18:42:07Z"
}
```

5. Data-flow diagram



The system has two flows. **Flow A** runs once per set: strain becomes a raw count, then a side weight; motion becomes rep events; the phone assembles and confirms the set; the account service stores it; the dashboard shows it. **Flow B** runs once per lifter per bar: the tag gives the phone the bar ID and nonce, the lifter signs in, the account service opens the only session allowed on that bar, and the phone binds both rings to it.

Timing budgets from the requirements

Budget	Where	Source
Rings connected ≤ 5 s after the tap	P8 → P11	AC-01.2
Side weight sent only after the bar is still ≥ 1 s	P2	UC-01 step 4
Set closes 10 s after the last rep; summary on screen ≤ 13 s after racking	P4	UC-01 step 8, AC-01.1
Set confirmed after 12 s of bar idle, or sooner if the lifter accepts	P5	UC-02 E1
Set saved to the account ≤ 10 s after confirm	P6	AC-01.1
New lifter's tap answered "bar in use" ≤ 2 s if the bar has been idle < 12 s; handover ≤ 5 s otherwise	P10	AC-02.2, AC-02.3
10 min with no motion → session ends, bar sleeps	P10, C2	AC-02.1

6. Architecture pattern and justification

We use a combination of four patterns, each governing one part of the system:

Part of the system	Pattern	Why it fits that part
Each sensor ring (C1, C2, C3 on X1)	**Embedded sensor-actuator**	The ring senses strain and motion, decides locally (still or moving, rep or not, sleep or wake), and acts by sleeping, waking and sending events. It must work alone on a small battery.
Measurement path, P1 → P7	**Pipeline**	Data only moves one way and changes form at each stage: strain → count → side weight → set → stored row → staff table. Each stage can be tested alone with recorded input.
Phone app, account service, dashboard (C4, C5, C6)	**Client-server**	Many phones and staff browsers share one record. The server is the single authority for "one open session per bar," which US-04 requires.
Inside the account service (C5)	**Layered** (API → rules → storage)	Keeps the session and duplicate-set rules out of the API code and the database code; detailed in D1.

Against the five criteria

Criterion	How the combination meets it
Fit to the problem	Weight and reps come from hardware that has to run untethered, so the rings are embedded nodes. The set data has a natural one-way path, so a pipeline. Sessions must never overlap across phones, so one server owns them.
Team skills	Two computer science students. Embedded C/C++ for the rings and a web-style client and REST server are both within our experience. No distributed-systems operations are needed.
Performance and timing	Weight and reps are computed on the ring (P2, P3), so Bluetooth carries a few small events per rep instead of a raw stream. That keeps the ≤ 1 s weight and ≤ 13 s summary budgets with margin. One server decision per tap keeps handover ≤ 5 s.
Scalability	One rec center has about 10–30 bars and a few hundred lifters. Rings are independent, and a stateless service can run more copies if a second gym is added.
Hardware constraints	The ring has no Wi-Fi, a 96 mm housing and a battery that must last weeks, so it sleeps at tens of µA and only talks over Bluetooth LE. The phone is therefore the gateway to the cloud.

Our Week 3 constraints are built in: the sprint budget is about $500, so we use off-the-shelf boards (X1) and a hosted platform (X4) rather than custom servers. For privacy, C5 is the only writer of set records, staff see data only by role (I11 returns 403), and no passwords are stored (X3). For accessibility, alerts on I7 use vibration as well as the screen.

Rejected alternatives

- **Microservices** (separate session, set and bar services). Rejected: three deployable services is more operations work than two people can support. It adds network hops inside the ≤ 10 s save budget, and our load does not need independent scaling.
- **Ring talks directly to the cloud over Wi-Fi** (IoT publish-subscribe). Rejected: a Wi-Fi radio draws far more current than Bluetooth LE and would drain the ring battery in days. Gym Wi-Fi often needs a sign-in page. The bar also cannot know who is lifting without the lifter's phone.
- **Master ring** (left ring sends to right ring, right ring talks to the phone). Rejected: it adds a second radio link and firmware state, and if the right ring fails both sides are lost. With each ring talking to the phone directly, the phone simply adds the two sides.

7. Decision log

#	Decision	Alternatives considered	Why the chosen option won
DL-1	Measure weight with shear strain gauges bonded to the bar shaft just inside each collar	Load-cell collars; pressure pads under the plates; RFID tags on each plate; lifter types the weight	Works with any plates the gym already owns. Nothing sits under the plates, so the bar can be dropped. Shear inside the collar measures only the weight outside it, whatever the bar's position.
DL-2	Put a thin 18 mm ring in the gap between the collar and the J-hook	Ring next to the J-hooks; housing at the bar center; electronics inside the sleeve	The gauges must sit outside every support (J-hooks, hands). Away from the center they stay out of the grip. 18 mm fits any rack with ≥ 22 mm of gap. The sleeve spins, so it cannot hold wired sensors.
DL-3	Sign in with an NFC tap on the bar	QR code on the bar; automatic Bluetooth proximity; PIN typed on the phone	A tap is quick and works with chalky hands. It proves the phone is physically at that bar, which Bluetooth proximity cannot do in a crowded gym. The tag's nonce lets the bar reject an old link (US-04).
DL-4	Each ring connects to the phone directly; the phone adds the two sides	One master ring relays the other; a cable along the bar	No ring-to-ring radio link to debug, no cable to snag. One ring failing still leaves the other side readable.
DL-5	Weight, reps and speed are computed on the ring; the phone receives events	Stream raw sensor data to the phone and compute there; compute in the cloud	Uses far less radio time (battery), still works if the phone connection drops (sets buffered on the ring), and keeps Bluetooth traffic small enough for the timing budgets.
DL-6	The account service owns sessions (one open session per bar)	The ring decides who owns it; the phone decides	Only the server sees every phone, so only it can enforce "never two lifters on one bar" (US-04, UC-02). The ring would need accounts and clocks it does not have.
DL-7	Sign-in through an external identity provider	Our own usernames and passwords	We store no passwords (privacy), and students already have campus or Google accounts, so there is no new account to create.
DL-8	Sleep-first power with wake on motion or NFC field, USB-C charging on each ring	Always-on radio; a separate charging stand; replaceable coin cells	Bars sit idle about 99% of the time, so sleeping gives weeks per charge. A stand was ruled out by the gym-floor requirement, and coin cells cannot supply the radio's peak current.